

High-Fidelity Closed-Loop Low-Thrust Trajectory Design

with a Quantum-Ready Binary Scheduler

GMAT-in-the-loop QUBO optimization and a SMART-1 case study

Gino Luciano Moretta^{*1} and Antonio Paolozzi¹

¹School of Aerospace Engineering, Sapienza Università di Roma

June 2026 — working note (v3)

Abstract

We extend our binary-schedule QUBO transcription of low-thrust trajectory design from an in-house planar circular restricted three-body (CR3BP) propagator to a *high-fidelity closed loop* in which NASA’s General Mission Analysis Tool (GMAT) plays the role of the truth model. A Python layer orchestrates the loop: it writes GMAT mission scripts, runs them headless, measures each candidate burn’s effect on the spacecraft state by finite differences *within the real dynamics*, assembles a quadratic unconstrained binary optimization (QUBO) over the engine’s on/off schedule, solves it, and validates the chosen schedule back in GMAT under a trust-region accept rule. The QUBO is solved classically here, but it is the exact form a quantum annealer ingests, so the combinatorial step is hardware-swappable with no change to the formulation. We exercise the loop on ESA’s SMART-1 electric-propulsion transfer, starting from its published geostationary transfer orbit (GTO) and Hall thruster: the orbit-raising budget matches the flown mission, the optimizer rediscovers the Oberth perigee-thrust strategy on its own under a duty-cycle constraint, and a launch-epoch phase search produces a real lunar encounter that the binary scheduler brakes into a bound lunar orbit. We characterise the computational profile (dominated by high-fidelity propagation, linear in the number of decision slots, not exponential), explain why the loop never simulates the 2^N burn combinations, and close with an applied outlook: autonomous on-board maneuver planning and a concrete path to quantum hardware for large, dense scheduling problems.

Keywords: low-thrust trajectory optimization, QUBO, quantum annealing, GMAT, sequential linearization, trust region, lunar capture, SMART-1, autonomous guidance.

1 Introduction and relation to the previous work

A companion note (v2) posed low-thrust cislunar trajectory correction as a *naturally binary* QUBO: at each of N time steps an on/off command $q_i \in \{0, 1\}$ decides whether a constant-thrust engine fires, and the schedule that best reaches a target is the ground state of a QUBO assembled from a first-order state-transition-matrix expansion. That work was developed and validated entirely in an *in-house* planar CR3BP propagator: a fast, transparent model, ideal for deriving the method and proving its monotonicity, but an idealised one (planar, nondimensional, three point masses, fixed-step RK4).

The natural and most frequent objection to any such result is: *would the schedule survive real dynamics?* This note answers it by replacing the in-house propagator with NASA GMAT [2] as

^{*}morettaginoluciano@gmail.com

the truth model — full DE-series ephemerides, Earth gravity field, Moon and Sun, finite burns, mass depletion — while keeping the same binary-QUBO decision layer. The result is a closed *design*→*simulate*→*read*→*decide* loop, the automated analogue of the write–run–read–fix cycle an engineer uses, with a high-fidelity simulator in the loop rather than a toy model.

The contributions of this note are:

1. A high-fidelity closed-loop architecture (Sec. 2) in which Python orchestrates GMAT and a QUBO solver, with the impulse-response sensitivities measured by finite differences in the truth model itself.
2. A clear account (Sec. 3) of why the loop is tractable — why it never simulates the 2^N burn combinations — which is the crux of any linearization-based optimizer.
3. A SMART-1 case study (Sec. 4) from the real launch orbit: orbit-raising budget, a self-discovered Oberth strategy, a real lunar encounter, and a binary-scheduled capture, with honest limits.
4. A computational and quantum-readiness analysis (Sec. 5) and an applied outlook (Sec. 6) on on-board autonomy and quantum hardware.

2 Method: a high-fidelity closed loop

Three components, with disjoint roles, exchange data in a loop (Fig. 1).

GMAT — the truth. GMAT never optimizes. It answers one question exactly: *given this on/off firing pattern, what is the resulting state?* We script a spacecraft with an electric thruster and a force model (Earth 4×4 gravity, Moon and Sun as point masses, DE405 ephemerides), run it headless via a subprocess, and parse the report file.

Python — the orchestrator and the brain. Python templates the GMAT scripts, launches the runs (in parallel), reads the results, assembles the QUBO, solves it, and applies the accept/reject logic.

The QUBO — the combinatorial decision. A quadratic binary objective whose minimiser is the firing pattern. It is solved here by exact enumeration (for $N \leq 20$) or simulated annealing, but it is written in the canonical form $\min_{\mathbf{q} \in \{0,1\}^N} \mathbf{q}^\top \mathbf{Q} \mathbf{q} + \boldsymbol{\ell}^\top \mathbf{q}$ that a quantum annealer accepts natively.

2.1 The binary scheduling QUBO

Over one maneuver window of duration T split into N equal slots, let ΔE_j be the change in a scalar objective (here the Moon-relative specific energy, or more generally a weighted final-state miss) produced by firing *only* slot j , measured in GMAT. The first-order model assumes the slot effects superpose,

$$E(\mathbf{q}) \approx E_0 + \sum_{j=1}^N q_j \Delta E_j, \quad (1)$$

and the schedule that drives E to a target $E_\star$ with the fewest burns is the minimiser of

$$f(\mathbf{q}) = \left(\sum_j q_j \Delta E_j - d \right)^2 + \alpha \sum_j q_j, \quad d = E_\star - E_0, \quad (2)$$

which expands to $\mathbf{q}^\top \mathbf{Q} \mathbf{q} + \boldsymbol{\ell}^\top \mathbf{q} + \text{const}$ with $\mathbf{Q} = \Delta \mathbf{E} \Delta \mathbf{E}^\top$ and $\boldsymbol{\ell} = \alpha \mathbf{1} - 2d \Delta \mathbf{E}$. The fuel weight α is a small tiebreaker; the energy term must dominate it, or the optimizer refuses to brake.

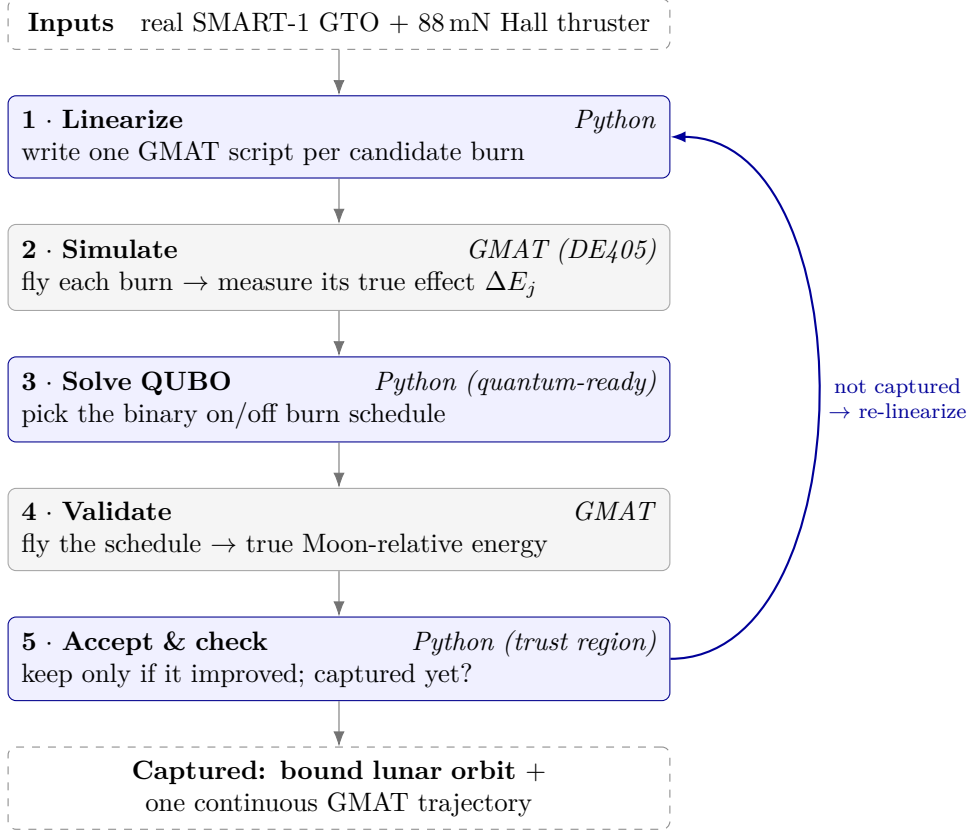


Figure 1: The closed loop. Blue = Python (orchestrator + QUBO optimizer); grey = GMAT (high-fidelity simulator, the “truth”). Steps 1–2 build a model from real physics; step 3 makes the combinatorial decision; steps 4–5 validate it and decide whether to repeat. Phase 1 (the GTO→Moon spiral) uses the same write–run–read loop with continuous thrust.

2.2 The outer loop: sequential re-linearization with a trust region

Equation (1) is only first-order. After the QUBO returns a candidate $\hat{\mathbf{q}}$, we fly the *actual* schedule in GMAT and read the true energy. The candidate is accepted only if the true miss improved; the loop then re-linearises around the new schedule and repeats, terminating at a fixed point or when no candidate improves. This is the binary analogue of sequential convex programming with a trust-region safeguard, and it is monotone: it can never return a schedule worse than coasting. For the near-parabolic SMART-1 capture a single window suffices; long arcs are handled by chaining short windows (a receding-horizon / model-predictive structure).

3 Why the loop never simulates 2^N combinations

It is tempting to think that choosing among 2^N firing patterns requires simulating them. It does not, and this is the heart of the method.

We measure each burn’s effect *individually* — N single-slot GMAT runs — and the QUBO *assumes the effects add* (Eq. 1). The true dynamics couple the burns (firing slot A shifts the state, so slot B acts slightly differently), and that coupling is *not* measured; it is the modelling error. The safeguard is the outer loop: we fly the one chosen combination in GMAT (step 4) and verify it; if superposition was too optimistic, the trust-region rule rejects the step or we re-linearise. Hence

N runs to build the model + 1 run to validate the chosen combination, per iteration — never 2^N .

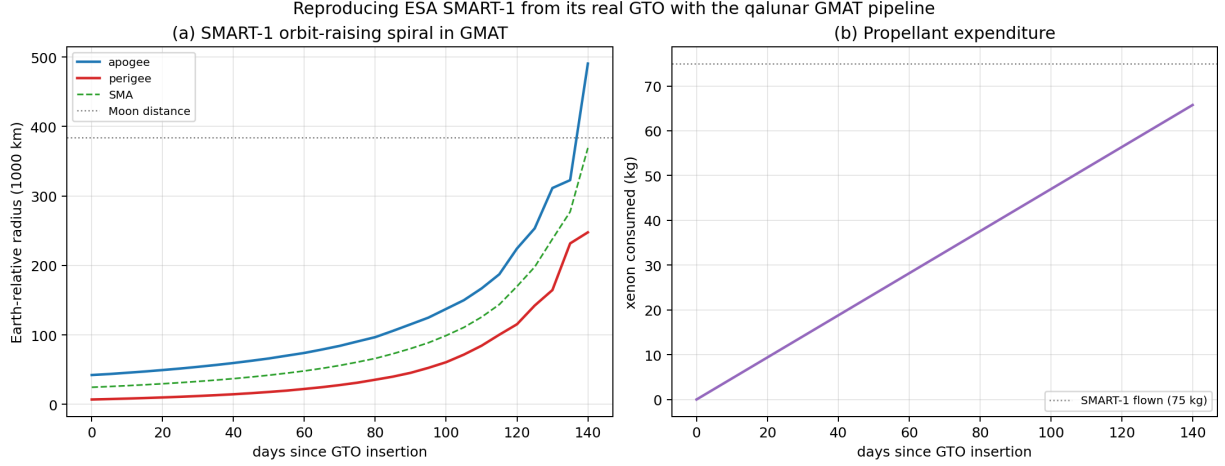


Figure 2: SMART-1 orbit-raising spiral reproduced in GMAT from the real GTO. *Left:* apogee/perigee/SMA climbing toward lunar distance. *Right:* xenon expended (linear), with the flown total marked.

The exponential search lives entirely in the *cheap* QUBO: evaluating the quadratic form 2^N times costs microseconds, not GMAT propagations. The expensive, high-fidelity simulator is touched only $N+1$ times per pass plus one validation. This is the classic bargain of linearization-based optimization — measure the pieces, assume they compose, verify the answer — and the trust region is what makes “assume they compose” safe.

4 Case study: re-flying SMART-1 from its real GTO

ESA’s SMART-1 reached the Moon by spiralling out of a geostationary transfer orbit with a single PPS-1350 Hall thruster over ~ 14 months [5, 6]. We take its published inputs: GTO $654 \times 35\,885$ km altitude, $i = 7^\circ$; Hall thruster 88 mN, $I_{sp} = 1650$ s; wet mass 367 kg, 82 kg xenon. Everything below runs in GMAT R2025a under real ephemerides (Earth 4×4 , Moon, Sun).

4.1 Phase 1 — the orbit-raising spiral

Continuous tangential thrust raises the apogee from GTO to lunar distance in 140 d at 100 % duty, costing $\Delta v \approx 3180$ m s $^{-1}$ and 65.8 kg of xenon, with the inclination essentially held ($7.0^\circ \rightarrow 6.42^\circ$). SMART-1 flew ≈ 3500 m s $^{-1}$ over ~ 410 d (including capture) at ~ 40 % duty; at that duty our 140 d becomes ~ 350 d. The agreement is the physics check: the orbit-raising Δv is the (thrust-independent) Edelbaum invariant (Fig. 2) [4].

4.2 The optimizer rediscovers the Oberth effect

SMART-1 could fire only ~ 40 % of each orbit (power, eclipse, thermal limits), so the operative question is *which arc* to thrust. We split one GTO orbit into $N = 24$ time slots, measure each slot’s energy gain ΔE_j in GMAT, and pose “raise energy with a budget of $K = \lfloor 0.4N \rfloor = 10$ burns” as a QUBO. The measured ΔE_j is linear in the slot speed — the Oberth law $dE = v dv$, recovered empirically — and the optimizer, with no orbital-mechanics knowledge, clusters all ten burns at perigee (Fig. 3). Validated in GMAT, the perigee schedule raises apogee 120 km per orbit against 64 km for evenly-spread firing ($1.9\times$) and 9.8 km for apogee-clustered firing — the same maximum-efficiency strategy SMART-1 flew operationally, derived here from the discrete side. (The dense, all-to-all cardinality QUBO is, notably, hard enough that classical simulated annealing did not reach its exact ground state at $N = 24$, motivating exact or quantum solvers.)

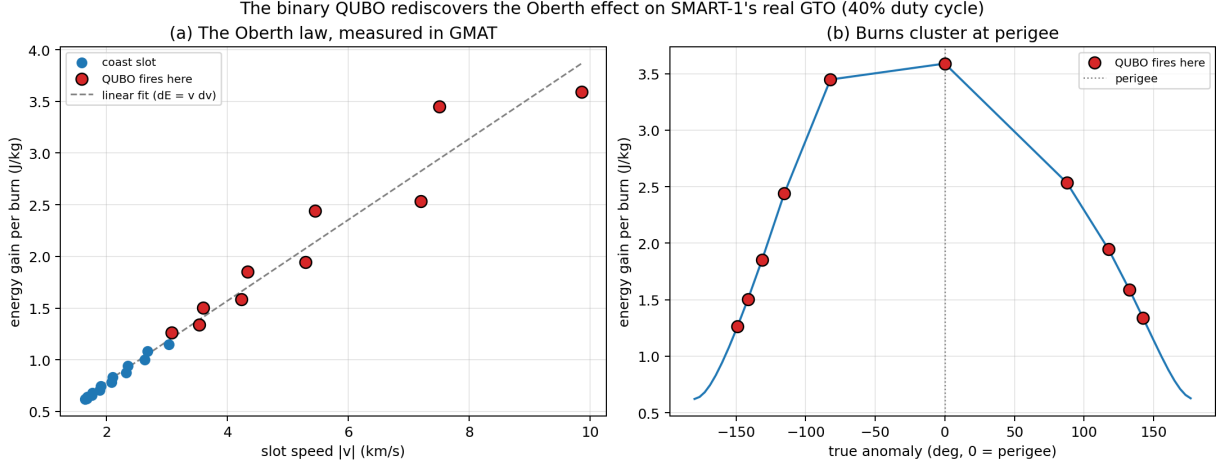


Figure 3: Under a 40% duty constraint the binary optimizer rediscovers Oberth. (a) measured energy gain per burn is linear in slot speed ($dE = v dv$). (b) the chosen burns cluster at perigee.

4.3 A real lunar encounter by phase search

The spiral is deterministic; whether it meets the Moon depends on the *phase* — where the Moon is when the apogee reaches lunar distance. We scan the launch epoch across a synodic month; the closest pass enters the lunar sphere of influence (SOI) at a near-parabolic perilune ≈ 7000 km from the Moon's centre, Moon-relative speed $\approx 1.21 \text{ km s}^{-1}$, specific energy $\approx +0.033 \text{ km}^2 \text{ s}^{-2}$ (just hyperbolic). The spiral thus delivers the spacecraft to the Moon almost on the capture boundary.

4.4 Binary-scheduled capture

Over a window straddling the SOI passage, the thruster is pointed anti-tangentially *relative to the Moon*; the QUBO chooses which slots fire to drive the Moon-relative energy negative. It selects burns clustered at perilune (Oberth again, now for braking) and binds the orbit: $E: +0.022 \rightarrow -0.006 \text{ km}^2 \text{ s}^{-2}$ for 41 m s^{-1} of braking. A more aggressive window ($\sim 63 \text{ m s}^{-1}$, the full single-pass authority) tightens it to $E = -0.0195 \text{ km}^2 \text{ s}^{-2}$, semi-major axis $\approx 125\,700$ km.

Honest limit. A single perilune pass yields only a *loose* bound orbit (apolune outside the SOI); under Earth's perturbation it drifts back out over weeks. Lowering it to an operational science orbit requires the multi-pass resonant capture that took the real SMART-1 team months. The loop surfaces this limit (it reports the residual positive apolune) rather than hiding it — the simulator keeps the method honest about what is genuinely hard.

5 Computational profile and quantum readiness

Where the cost is. The dominant cost is *high-fidelity propagation*, not optimization. Per outer iteration the loop performs $N+1$ GMAT runs (embarrassingly parallel) plus one validation; the QUBO solve is microseconds to seconds. The full SMART-1 design-and-fly loop runs in ~ 16 s of wall time before the GMAT visualization launch. Cost scales *linearly* in the number of slots N and in the number of windows, not exponentially.

A cheap-vs-faithful dial. The sensitivities ΔE_j can be built either from the cheap in-house CR3BP linearization (milliseconds, no GMAT) with GMAT used only to validate, or by finite differences in GMAT itself (seconds $\times N$, but exact to the truth model). The two ends trade speed for fidelity and can be mixed.

Quantum readiness, stated honestly. The QUBO is solved classically here; we have no quantum-processing-unit (QPU) access at present. The formulation, however, is *native* to a quantum annealer: N binary decisions map to N qubits with no encoding overhead beyond hardware connectivity, and only step 3 of the loop would change — the QUBO is submitted to the annealer and low-energy samples return in microseconds. GMAT, and physics generally, stay classical; a *fully “quantum” trajectory optimizer does not exist*, because an annealer solves combinatorial optimization, not differential equations. The method is therefore hybrid by nature: quantum for the discrete burn decision, classical for the dynamics.

6 Functional outlook: why this matters

Beyond reproducing a flown mission, the architecture has properties that make it worth developing further.

On-board autonomous maneuver planning. The binary scheduler with a trust-region rule is cheap to evaluate and robust by construction (never worse than coasting). It is, in effect, a model-predictive controller for low-thrust guidance: from the live estimated state it can re-decide the next window’s burns without ground contact. This is directly relevant to autonomous station-keeping, lunar capture, and constellation maintenance, where round-trip light time and operations cost make ground-in-the-loop planning expensive. The same write–run–read loop, with a flight-grade fast propagator in place of GMAT, is an on-board guidance candidate.

Hardware-faithful by construction. Real electric thrusters are pulsed and duty-limited; a binary on/off schedule is their native command language, so no continuous solution must be discretised after the fact. The SMART-1 duty-cycle result is a concrete example: the formulation *is* the operational constraint.

A concrete path to quantum hardware. The combinatorial layer is swappable with no change to the formulation. The natural next step is to run the scheduling QUBO on D-Wave’s Leap solvers (hybrid and, with access, the Advantage QPU). The regime where this pays off is large N — many slots, many windows, multi-channel thrust — where the QUBO becomes a dense, frustrated problem and classical heuristics degrade (we already observe simulated annealing missing the exact optimum of the duty-cycle QUBO at $N = 24$). A measured crossover study against exact (MILP) and heuristic solvers quantifies where quantum hardware becomes the rational choice.

Validated against a real tool and a real mission. Because the truth model is NASA GMAT and the case study is ESA SMART-1 from its real launch orbit, the results are credible to a mission-design audience: not a toy, and explicit about its limits.

7 Conclusion

We placed a high-fidelity simulator inside the binary-QUBO trajectory-design loop: Python writes and runs GMAT, measures per-burn sensitivities in the real dynamics, lets a quantum-ready QUBO choose the on/off schedule, and validates it under a monotone trust-region rule — never simulating the 2^N combinations, only $N+1$ builds plus one validation per pass. On SMART-1 from its real GTO the loop matched the orbit-raising budget, rediscovered the Oberth perigee-thrust strategy with no built-in orbital mechanics, found a real lunar encounter by phase search, and captured into a bound lunar orbit — while honestly exposing that a held science orbit needs the multi-pass lowering that is the genuinely hard part. The architecture is computationally favourable (linear in decision steps, dominated by propagation), hybrid by nature, and points to

two developments worth pursuing: autonomous on-board maneuver planning, and the scheduling QUBO on quantum hardware for the large, dense problems where it should pay off.

Reproducibility

All results are produced by the open `qalunar` codebase against GMAT R2025a. The end-to-end SMART-1 design-and-fly pipeline is a single entry point (`scripts/run_smart1_demo.py`); the spiral, duty-cycle, encounter and capture experiments are separate scripts under `scripts/`.

References

- [1] De Grossi, F., Carbone, A., Spiller, D., Ottaviani, D., Mengoni, R., Circi, C., *Transcription and optimization of an interplanetary trajectory through quantum annealing*, *Astrodynamics* **9**:195–215, 2025.
- [2] NASA, *General Mission Analysis Tool (GMAT) R2025a*, open-source, <https://gmat.gsfc.nasa.gov>.
- [3] Glover, F., Kochenberger, G., Du, Y., *A tutorial on formulating and using QUBO models*, *4OR* **17**(4):335–371, 2019.
- [4] Edelbaum, T. N., *Propulsion Requirements for Controllable Satellites*, *ARS Journal* **31**(8):1079–1089, 1961.
- [5] Racca, G. D., et al., *SMART-1 mission description and development status*, *Planetary and Space Science* **50**(14–15):1323–1337, 2002.
- [6] Milligan, D., Camino, O., Gestal, D., *SMART-1 electric propulsion operations*, 41st AIAA/ASME/SAE/ASEE Joint Propulsion Conference, AIAA 2005-3671, 2005.
- [7] D-Wave Systems Inc., *Leap Quantum Cloud Service and Ocean SDK documentation*, <https://docs.dwavequantum.com>.